

CS 4348 Project 3 Report

Pranav Sawant

08 May 2026

1 Project Overview

The goal of this project was to implement a command-line program that creates and manages index files containing a B-tree. The program supports creating a new index file, inserting key/value pairs, searching for a key, loading many key/value pairs from a CSV file, printing all entries, and extracting the contents of the index into a CSV file. I implemented the project in Python in a file named `project3.py`.

The index file is treated as a binary file divided into fixed-size 512-byte blocks. Block 0 is reserved for the file header, and every B-tree node is stored in exactly one 512-byte block. All integer values written to the file are stored as 8-byte unsigned integers using big-endian byte order. The B-tree uses minimum degree 10, which means each node can contain up to 19 key/value pairs and 20 child pointers.

2 Development Context

Although the general project workflow encourages implementing the project across multiple sessions, I completed this project in one focused sitting. This was mainly because I had three finals during the same week, along with multiple project and assignment submissions. I also had prior experience working with B-trees in my Data Structures and Algorithms (CS 3345) and Database Systems (CS 4347) classes, so I was already familiar with the core B-tree operations such as search, node splitting, and insertion into non-full nodes.

Because of these constraints, I chose to plan the full implementation first and then build the program in one continuous session. This development process is reflected in both my `devlog.md` file and my Git commit history.

3 File Layout and Binary Format

The project required the index file to use fixed-size 512-byte blocks. I followed this structure exactly. The first block is the file header, and all later blocks are B-tree nodes.

3.1 Header Block

The header block stores the following information:

Field	Purpose
Magic number	Stores the ASCII sequence <code>4348PRJ3</code> to identify a valid index file.
Root block ID	Stores the block number of the root node. This value is 0 when the tree is empty.
Next block ID	Stores the block number that should be assigned to the next new node appended to the file.
Unused bytes	Fill the rest of the 512-byte header block.

This header design made it simple to validate files and track the current root and the next available block position.

3.2 Node Block

Each B-tree node is also stored in a 512-byte block. The node format is fixed and consists of the node's block ID, parent block ID, number of keys, an array of 19 keys, an array of 19 values, and an array of 20 child pointers. The remaining space in the block is unused.

The fixed-size layout allowed the program to read or write any node directly using its block ID. Since each block is 512 bytes, the byte offset of a node is simply:

$$\text{offset} = \text{block_id} \times 512$$

This avoids the need for variable-length records or complicated file scanning.

4 B-Tree Design

The B-tree uses minimum degree $t = 10$. Therefore, each node can store at most $2t - 1 = 19$ key/value pairs and at most $2t = 20$ child pointers.

The implementation follows the standard B-tree insertion algorithm:

1. If the tree is empty, create a new root node and insert the first key/value pair.
2. If the root is full, create a new root, split the old root, and then insert into the appropriate child.
3. If the current node is not full, insert the key/value pair in sorted order if the node is a leaf.

4. If the current node is internal, find the correct child. If that child is full, split it before descending.

This strategy ensures that insertion never descends into a full child node. As a result, the tree remains balanced and every node respects the maximum key limit.

5 Command Implementation

The program supports six commands: `create`, `insert`, `search`, `load`, `print`, and `extract`. Each command is given through command-line arguments.

5.1 Create

The `create` command creates a new index file. If the requested file already exists, the program prints an error and does not modify the file. A newly created index file contains only the header block. The root block ID is set to 0 because the tree is initially empty, and the next block ID is set to 1 because block 0 is already used by the header.

Example:

```
python3 project3.py create test.idx
```

5.2 Insert

The `insert` command inserts a key/value pair into the B-tree stored in the index file. The file must already exist and must contain the correct magic number. The key and value are converted to integers before insertion.

Example:

```
python3 project3.py insert test.idx 15 100
```

If the inserted key already exists, the program updates the existing value instead of inserting a duplicate key. I chose this behavior because it keeps the index unambiguous: each key maps to one current value.

5.3 Search

The `search` command searches for a key in the B-tree. If the key exists, the program prints the key/value pair. If the key is not found, the program prints an error message.

Example:

```
python3 project3.py search test.idx 15
```

The search operation follows the sorted keys inside each node and descends through the correct child pointer until it either finds the key or reaches a leaf.

5.4 Load

The `load` command performs bulk insertion from a CSV file. Each line of the CSV file must contain a comma-separated key/value pair. The program reads the file line by line and inserts each pair into the existing B-tree index file.

Example:

```
python3 project3.py load test.idx input.csv
```

This command is useful when many entries need to be inserted at once. It behaves like repeatedly calling `insert` for every line in the input CSV file.

5.5 Print

The `print` command prints every key/value pair in the index to standard output. I implemented this using an in-order traversal of the B-tree so that the entries are printed in sorted key order.

Example:

```
python3 project3.py print test.idx
```

5.6 Extract

The `extract` command writes every key/value pair in the B-tree to a new CSV file. If the output file already exists, the program prints an error and leaves the existing file unmodified.

Example:

```
python3 project3.py extract test.idx output.csv
```

Like `print`, this command uses in-order traversal, so the extracted CSV file is sorted by key.

6 Error Handling

The program checks for several types of errors:

- Missing or incorrect command-line arguments
- Unknown commands
- Attempting to create an index file that already exists
- Attempting to use a missing index file

- Attempting to use an invalid index file
- Invalid integer keys or values
- Missing CSV files for the `load` command
- Attempting to extract into an output file that already exists
- Malformed CSV lines during loading

These checks help prevent accidental file corruption and make the program safer to use from the command line.

7 Memory Management

The project specification states that the implementation should never have more than three nodes in memory at one time. My implementation follows this rule by reading and writing nodes directly from disk as needed instead of loading the entire B-tree into memory. The header is kept in memory while the file is open, but nodes are loaded only for specific operations such as searching, splitting, inserting, or traversing.

During insertion, the algorithm mainly works with the current node, a child node, and occasionally a newly created or split node. This matches the intended disk-based B-tree design, where the file stores the tree and memory is used only as a temporary workspace.

8 Challenges and Design Decisions

The most important challenge was implementing the B-tree directly on disk rather than using a normal in-memory tree. Since every node is stored as a fixed-size binary block, the program has to carefully convert between Python objects and the exact binary layout required by the project.

Another challenge was keeping the header synchronized with the file. Whenever a new node is appended, the next block ID has to be updated. Similarly, when the root changes after a split, the root block ID in the header has to be updated.

One design decision I made was to update the value when inserting a duplicate key. This avoids storing multiple entries for the same key and makes search behavior simple and deterministic.

Another design decision was to print and extract entries using sorted order. Since B-trees naturally support in-order traversal, this makes the output easier to inspect and verify.

9 Conclusion

This project implemented a disk-based B-tree index file manager in Python. The program supports all required commands and uses the required 512-byte block structure, header format, node format, big-endian integer storage, and minimum degree 10 B-tree layout. The implementation keeps the tree stored in the file rather than loading the full structure into memory, which matches the intended behavior of an index-file manager.

Even though the project was completed in one focused sitting due to finals and other coursework, the implementation was guided by prior experience with B-trees and careful planning before coding. This process is documented in `devlog.md` and reflected in the Git commit history.